

Mock Exam Automata

Wednesday 11 June 2025, 16:00 – 17:15, R10

Teacher: Hans van Heumen

Allowed resources:

No communication means like telephone, smartwatch, ...

But typically: laptop with EFIT + java + antlr + (offline) Javadoc + Python

Each assignment has to be submitted in a **separate folder**.

1. Grammar assignment (33p)

General remarks:

- if you think that the questions are not precise enough to solve it, then you can add your own assumptions and continue your work.
Please clearly write down your assumptions and the reason for adding it
- use the common rules for Separation of Concerns, such as: divide the tasks between the lexer, the parser, and the java/python actions; and avoid that a visitor does the tasks which should be done by its child visitors

To hand in

- a. grammar file (.g4)
make sure that the rules and tokens have meaningful names
- b. source code (java or python)
- c. input file (.txt),
which clearly shows the capabilities of your grammar aka. which clearly shows your own test cases
- d. generated output for that input file (**screen shot only**, no txt file)
- e. parse tree picture (.svg)
- f. optional: any further document describing your intelligence (if the files above wouldn't be sufficient)

```

grammar ExamGrammar;

prog: expr EOF ;

expr
    : expr ('*' | '/') expr # MulDiv
    | expr ('+' | '-') expr # AddSub
    | ('+' | '-') expr      # Unary
    | INT                  # Int
    | ID                   # Id
    | '(' expr ')'         # Parens
    ;

INT : [0-9]+ ;
ID  : [a-zA-Z_] [a-zA-Z_0-9]* ;
WS  : [ \t\r\n]+ -> skip ;

```

Table 1

Use the **ExamGrammar** from Table 1 as the starting point for all assignments that follow.

Assignment A (3p, 2p):

Q1. Add support for **variables** in above grammar.

Your grammar should be able to parse expressions such as:

```

a = 15;
b = 123;
sum = a + b;

```

Q2. Add a **print command** to above grammar.

This command should allow printing the value of an expression, particularly variables.

Sample usage: print sum. Expected result is 138.

Assignment B (8p, 8p, 6p, 3p, 3p):

Many programming languages, such as Python, allow you to assign values to multiple variables in a single statement.

Examples:

```

a, b = 4, 3
d, e, f = (3+d), (b*5), 42

```

In the first example, variables a and b are assigned simultaneously. After the statement, a has the value 4, and b has the value 3.

Clarifications and Simplifications:

- Your grammar must support any number of variables and expressions in a multiple assignment.
- We recommend using a **visitor pattern** for implementation.
- All parts of the assignment must follow from the grammar defined in Table 1.

Questions:

- Q1. Extend the grammar to support **multiple assignments** as described above.
- Q2. Implement the corresponding programming actions based on the grammar rules.
- Q3. Ensure that an **error message** is produced if the number of variables on the left-hand side does not match the number of expressions on the right-hand side.
- Q4. Make sure that a **single assignment** such as `a = 42` is also covered by this rule.
- Q5. Ensure that a statement like `a, b = b, a` (i.e. **value swapping**) works correctly.

2. Regular expression (9p)

Write regular expressions for the following languages on $\{0, 1\}$:

1. All strings ending in 10.
2. All strings not ending in 10.

Write your answer to a file.

3. Grammar (9p)

Find a left-linear grammar for the language $L((aaab^*ba)^*)$.

Write your answer to a file.

4. DFA (9p)

Give dfa's for the languages

1. $L = \{ab^4wb^2 : w \in \{a, b\}^*\}$.
2. $L = \{ab^na^m : n \geq 3, m \geq 2\}$.

Draw your solution on the official paper and hand in your paper.

-- end of exam --